

Sommaire

Introduction

1. Contexte et objectifs du projet

- 1.1 Contexte du projet
- 1.2 Objectifs pédagogiques
- 1.3 Périmètre et choix techniques

2. Environnement de travail

- 2.1 Machine virtuelle
- 2.2 Outils utilisés
- 2.3 Vérification de l'installation des outils

3. Présentation de l'application

- 3.1 Description fonctionnelle
- 3.2 Endpoints exposés
- 3.3 Choix technologiques

4. Mise en place et test de l'application en local

- 4.1 Création de l'application
- 4.2 Gestion des dépendances avec un environnement virtuel
- 4.3 Lancement de l'application en local
- 4.4 Validation du fonctionnement de l'application

5. Conteneurisation de l'application avec Docker

- 5.1 Création de l'image Docker
- 5.2 Construction de l'image Docker
- 5.3 Lancement du conteneur Docker
- 5.4 Validation du fonctionnement de l'application conteneurisée
- 5.5 Consultation des logs du conteneur

6. Orchestration locale avec Docker Compose

- 6.1 Présentation de Docker Compose
- 6.2 Définition du service applicatif
- 6.3 Lancement de l'application avec Docker Compose
- 6.4 Validation du fonctionnement de l'application orchestrée
- 6.5 Consultation des logs avec Docker Compose
- 6.6 Arrêt de l'application

7. Mise en place du cluster Kubernetes avec Kind

- 7.1 Vérification de l'outil Kind
- 7.2 Création du cluster Kubernetes local
- 7.3 Vérification du bon fonctionnement du cluster
- 7.4 Création et vérification du namespace
- 7.5 Chargement de l'image Docker dans le cluster Kind

8. Déploiement de l'application sur Kubernetes

- 8.1 Déploiement de l'application via un Deployment
- 8.2 Exposition de l'application via un Service Kubernetes
- 8.3 Accès à l'application déployée
- 8.4 Vérification de l'infrastructure Kubernetes

9. Bilan et bonnes pratiques Kubernetes

- 9.1 Gestion de la configuration et des variables d'environnement
- 9.2 Probes de santé (liveness et readiness)
- 9.3 Supervision et accès aux logs

Conclusion générale

Mise en œuvre d'une application conteneurisée avec Docker et déployée sur Kubernetes (Kubernitest)

- 1. Contexte et objectifs du projet

Ce projet a pour objectif de concevoir, conteneuriser puis orchestrer une application simple afin de démontrer la maîtrise des fondamentaux de Docker et Kubernetes, depuis un environnement de développement local jusqu'à un déploiement orchestré sur un cluster Kubernetes.

L'objectif n'est pas le développement applicatif avancé, mais la compréhension et la mise en œuvre des mécanismes de :

- conteneurisation,
- packaging applicatif,
- orchestration,
- déploiement reproductible.

Ce projet est conçu comme un **TP technique démonstratif**, pouvant être présenté dans un cadre pédagogique ou lors d'un entretien technique.

2. Environnement de travail

2.1 Machine virtuelle

Le projet est réalisé sur une machine virtuelle Linux afin de garantir un environnement isolé et reproductible.

Configuration de la VM :

- Système d'exploitation : Ubuntu 22.04 LTS
- Processeur : 2 vCPU
- Mémoire vive : 4 Go RAM
- Stockage : 40 Go
- Réseau : NAT
- Hyperviseur : (VMware)

Cette configuration est suffisante pour exécuter Docker et un cluster Kubernetes local via Kind.

3. Outils utilisés

Les outils suivants sont utilisés tout au long du projet :

- **Docker** : conteneurisation de l'application
- **Docker Compose** : orchestration locale multi-conteneurs
- **kubect**l : outil en ligne de commande pour interagir avec Kubernetes
- **Kind (Kubernetes IN Docker)** : création d'un cluster Kubernetes local

4. Vérification de l'installation des outils

Avant de démarrer le projet, il est nécessaire de vérifier que tous les outils sont correctement installés et fonctionnels.

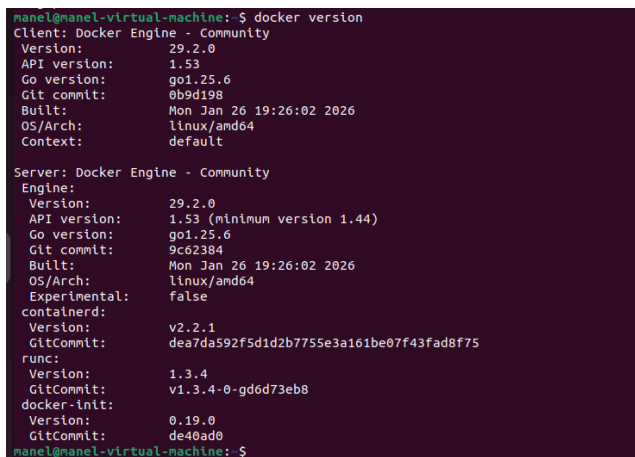
4.1 Vérification de Docker

Commande exécutée :

docker version

Cette commande permet de vérifier :

- la présence du client Docker,
- le bon fonctionnement du Docker Engine,
- la version utilisée (client et serveur).

A screenshot of a terminal window showing the output of the 'docker version' command. The output is divided into two main sections: 'Client: Docker Engine - Community' and 'Server: Docker Engine - Community'. The client section lists version 29.2.0, API version 1.53, Go version go1.25.6, Git commit 0b9d198, built on Mon Jan 26 19:26:02 2026, OS/Arch linux/amd64, and context default. The server section lists version 29.2.0, API version 1.53 (minimum version 1.44), Go version go1.25.6, Git commit 9c62384, built on Mon Jan 26 19:26:02 2026, OS/Arch linux/amd64, Experimental: false, containerd version v2.2.1, GitCommit dea7da592f5d1d2b7755e3a161be07f43fad8f75, runc version 1.3.4, GitCommit v1.3.4-0-gd6d73eb8, docker-init version 0.19.0, and GitCommit de40ad0. The prompt 'manel@manel-virtual-machine: \$' is visible at the top and bottom of the terminal output.

```
manel@manel-virtual-machine: $ docker version
Client: Docker Engine - Community
 Version:           29.2.0
 API version:       1.53
 Go version:        go1.25.6
 Git commit:        0b9d198
 Built:             Mon Jan 26 19:26:02 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:           29.2.0
  API version:       1.53 (minimum version 1.44)
  Go version:        go1.25.6
  Git commit:        9c62384
  Built:             Mon Jan 26 19:26:02 2026
  OS/Arch:           linux/amd64
  Experimental:      false
 containerd:
  Version:           v2.2.1
  GitCommit:        dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc:
  Version:           1.3.4
  GitCommit:        v1.3.4-0-gd6d73eb8
 docker-init:
  Version:           0.19.0
  GitCommit:        de40ad0
manel@manel-virtual-machine: $
```

capture d'écran de la sortie de la commande docker version.

4.2 Test de Docker avec une image officielle

Commande exécutée :

docker run hello-world

Cette commande télécharge l'image officielle hello-world depuis Docker Hub et exécute un conteneur de test.

L'affichage du message *"Hello from Docker!"* confirme que :

- Docker est capable de télécharger une image,
- un conteneur peut être lancé correctement.

```
manel@manel-virtual-machine:~$ docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bdc9d7: Pull complete
ea52d2080f99: Download complete
Digest: sha256:05813aedc15fb7b4d732e1be879d3252c1c9c25d885824f6295cab4538cb85cd
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the 'hello-world' image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
manel@manel-virtual-machine:~$
```

capture d'écran du message de succès Hello from Docker!.

4.3 Vérification de kubectl

Commande exécutée :

kubectl version --client

Cette commande permet de vérifier que l'outil kubectl est bien installé sur la machine cliente et prêt à interagir avec un cluster Kubernetes.

```
manel@manel-virtual-machine:~$ kubectl version --client
Client Version: v1.30.14
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
manel@manel-virtual-machine:~$
```

capture d'écran de la version client de kubectl.

4.4 Vérification de Kind

Commande exécutée :

kind version

Kind est utilisé pour créer un cluster Kubernetes local exécuté dans des conteneurs Docker. Cette commande confirme que l'outil est correctement installé.

```
manel@manel-virtual-machine:~$ kind version
kind v0.23.0 go1.21.10 linux/amd64
manel@manel-virtual-machine:~$
```

capture d'écran affichant la version de Kind.

-
- déployer l'application sur un cluster Kubernetes via des manifests YAML.

5. Mise en place et test de l'application en local

5.1 Création de l'application

Une application web simple a été développée en **Python** à l'aide du framework **Flask**. Ce choix s'explique par la légèreté du framework et sa facilité d'intégration dans un environnement conteneurisé.

L'application expose deux endpoints HTTP :

- / : retourne un message confirmant le bon fonctionnement de l'application
- /health : endpoint de santé retournant l'état de l'application

Ces endpoints serviront par la suite à vérifier le bon fonctionnement du conteneur et à configurer les probes Kubernetes.

5.2 Gestion des dépendances avec un environnement virtuel

Afin d'isoler les dépendances du projet, un **environnement virtuel Python (venv)** a été utilisé.

Sous Ubuntu, les modules python3-venv et pip ne sont pas installés par défaut. Leur installation est nécessaire pour pouvoir créer un environnement virtuel et gérer les dépendances Python du projet.

Commandes exécutée :

```
sudo apt update
```

```
sudo apt install -y python3-venv python3-pip
```

```
Setting up libjs-jquery (3.6.0+dfsg+~3.5.13-1) ...
Setting up libbinutils:amd64 (2.38-4ubuntu2.12) ...
Setting up libc-dev-bin (2.35-0ubuntu3.12) ...
Setting up libalgorithm-diff-xs-perl (0.04-6build3) ...
Setting up libcc1-0:amd64 (12.3.0-1ubuntu1-22.04.2) ...
Setting up liblsan0:amd64 (12.3.0-1ubuntu1-22.04.2) ...
Setting up libitm1:amd64 (12.3.0-1ubuntu1-22.04.2) ...
Setting up libc-devtools (2.35-0ubuntu3.12) ...
Setting up libjs-underscore (1.13.2-dfsg-2) ...
Setting up libalgorithm-merge-perl (0.08-3) ...
Setting up libtsan0:amd64 (11.4.0-1ubuntu1-22.04.2) ...
Setting up libctf0:amd64 (2.38-4ubuntu2.12) ...
Setting up python3.10-venv (3.10.12-1-22.04.13) ...
Setting up python3-venv (3.10.6-1-22.04.1) ...
Setting up libjs-sphinxdoc (4.3.2-1) ...
Setting up libgcc-11-dev:amd64 (11.4.0-1ubuntu1-22.04.2) ...
Setting up libc6-dev:amd64 (2.35-0ubuntu3.12) ...
Setting up binutils-x86_64-linux-gnu (2.38-4ubuntu2.12) ...
Setting up binutils (2.38-4ubuntu2.12) ...
Setting up dpkg-dev (1.21.1ubuntu2.6) ...
Setting up libexpat1-dev:amd64 (2.4.7-1ubuntu0.6) ...
Setting up libstdc++11-dev:amd64 (11.4.0-1ubuntu1-22.04.2) ...
Setting up zlib1g-dev:amd64 (1:1.2.11.dfsg-2ubuntu9.2) ...
Setting up gcc-11 (11.4.0-1ubuntu1-22.04.2) ...
Setting up g++-11 (11.4.0-1ubuntu1-22.04.2) ...
Setting up gcc (4:11.2.0-1ubuntu1) ...
Setting up libpython3.10-dev:amd64 (3.10.12-1-22.04.13) ...
Setting up python3.10-dev (3.10.12-1-22.04.13) ...
Setting up g++ (4:11.2.0-1ubuntu1) ...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mode
Setting up build-essential (12.9ubuntu3) ...
Setting up libpython3-dev:amd64 (3.10.6-1-22.04.1) ...
Setting up python3-dev (3.10.6-1-22.04.1) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.12) ...
manel@manel-virtual-machine:~/kubernittest/app$
```

Capture d'écran de l'installation des paquets python3-venv et python3-pip.

Une fois les outils installés, l'environnement virtuel a été créé et activé, puis les dépendances ont été installées à partir du fichier requirements.txt.

```
(.venv) manel@manel-virtual-machine:~/kubernitest/app$ pip install -r requirements.txt
Collecting Flask==3.0.3
  Downloading flask-3.0.3-py3-none-any.whl (101 kB)
    101.7/101.7 KB 1.3 MB/s eta 0:00:00
Collecting Jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
    134.9/134.9 KB 8.3 MB/s eta 0:00:00
Collecting click>=8.1.3
  Downloading click-8.3.1-py3-none-any.whl (108 kB)
    108.3/108.3 KB 22.4 MB/s eta 0:00:00
Collecting blinker>=1.6.2
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting itsdangerous>=2.1.2
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting Werkzeug>=3.0.0
  Downloading werkzeug-3.1.5-py3-none-any.whl (225 kB)
    225.0/225.0 KB 33.1 MB/s eta 0:00:00
Collecting MarkupSafe>=2.0
  Downloading markupsafe-3.0.3-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Installing collected packages: MarkupSafe, itsdangerous, click, blinker, Werkzeug, Jinja2, Flask
Successfully installed Flask-3.0.3 Jinja2-3.1.6 MarkupSafe-3.0.3 Werkzeug-3.1.5 blinker-1.9.0 click-8.3.1 itsdangerous-2.2.0
(.venv) manel@manel-virtual-machine:~/kubernitest/app$
```

Capture d'écran montrant l'activation de l'environnement virtuel et l'installation des dépendances.

5.3 Lancement de l'application en local

L'application a ensuite été lancée en local à l'aide de l'interpréteur Python. Des variables d'environnement ont été utilisées pour définir :

- le message retourné par l'application,
- le port d'écoute du serveur.

```
(.venv) manel@manel-virtual-machine:~/kubernitest/app$ cd ~/kubernitest/app
source .venv/bin/activate
export APP_MESSAGE="Hello Kubernitest"
export PORT=5000
python src/app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.242.132:5000
Press CTRL+C to quit
127.0.0.1 - - [28/Jan/2026 16:01:28] "GET /health HTTP/1.1" 200 -
```

Capture d'écran montrant le démarrage du serveur Flask et l'écoute sur le port 5000.

5.4 Validation du fonctionnement de l'application

Le bon fonctionnement de l'application a été validé à l'aide de requêtes HTTP effectuées avec l'outil curl sur un deuxième terminal.

- L'endpoint / retourne correctement le message attendu.
- L'endpoint /health retourne un statut indiquant que l'application est opérationnelle.

```
manel@manel-virtual-machine:~$ curl -s http://127.0.0.1:5000/  
curl: command not found  
manel@manel-virtual-machine:~$ curl -s http://127.0.0.1:5000/health  
{"status":"ok"}  
manel@manel-virtual-machine:~$ ~  
bash: /home/manel: Is a directory  
manel@manel-virtual-machine:~$ curl -s http://127.0.0.1:5000/  
curl -s http://127.0.0.1:5000/health  
{"message":"Hello Kubernitest"}  
manel@manel-virtual-machine:~$ curl -s http://127.0.0.1:5000/health  
{"status":"ok"}  
manel@manel-virtual-machine:~$
```

Capture d'écran de la requête vers l'endpoint /.

Capture d'écran de la requête vers l'endpoint /health.

Ces tests confirment que l'application fonctionne correctement en local avant sa conteneurisation.

6. Conteneurisation de l'application avec Docker

6.1 Création de l'image Docker

Afin de conteneuriser l'application, un fichier **Dockerfile** a été créé.

Ce fichier définit les différentes étapes nécessaires à la construction de l'image Docker contenant l'application Python.

Le Dockerfile repose sur une image de base officielle Python, puis :

- définit un répertoire de travail,
- installe les dépendances à partir du fichier requirements.txt,
- copie le code source de l'application,
- expose le port utilisé par l'application,
- définit la commande de démarrage du conteneur.

6.2 Construction de l'image Docker

L'image Docker a été construite à l'aide de la commande `docker build`.
Lors d'une première tentative, une erreur a été rencontrée en raison de l'absence du fichier Dockerfile dans le répertoire courant.

```
cd: command not found
(.venv) manel@manel-virtual-machine:~/kubernitest$ docker build -t kubernitest-app:1.0 .
[+] Building 0.5s (1/1) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.2s
=> => transferring dockerfile: 2B                               0.0s
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
```

Capture d'écran montrant l'erreur Docker indiquant l'impossibilité de trouver le fichier Dockerfile.

Cette erreur a permis de rappeler que Docker recherche par défaut un fichier nommé Dockerfile dans le répertoire courant.
Après s'être positionné dans le bon dossier, la construction de l'image s'est déroulée correctement.

6.3 Lancement du conteneur Docker

Une fois l'image construite, un conteneur a été lancé à partir de celle-ci.
Le port 5000 du conteneur a été exposé vers le port 5000 de la machine hôte afin de permettre l'accès à l'application.

Une variable d'environnement a également été passée au conteneur afin de personnaliser le message retourné par l'application.

```
(.venv) manel@manel-virtual-machine:~/kubernitest/app$ docker run --rm -d --name kubernitest-app -p 5000:5000 \
-e APP_MESSAGE="Hello Kubernitest (Docker)" \
kubernitest-app:1.0
a51a1bf50503cd2e46e6b872fab0a17f493e8ce2bf1698d6742521d39bee7ee2
(.venv) manel@manel-virtual-machine:~/kubernitest/app$ docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS
PORTS
a51a1bf50503   kubernitest-app:1.0 "python src/app.py"      8 seconds ago Up 7
seconds       0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   kubernitest-app
(.venv) manel@manel-virtual-machine:~/kubernitest/app$
```

Capture d'écran de la commande `docker run` et de la liste des conteneurs actifs **via `docker ps`**.

Cette commande confirme que le conteneur est en cours d'exécution et que le port est correctement exposé.

6.4 Validation du fonctionnement de l'application conteneurisée

Le bon fonctionnement de l'application a été vérifié une nouvelle fois, cette fois-ci dans un environnement conteneurisé.

Les endpoints `/` et `/health` répondent correctement lorsque l'application est exécutée dans le conteneur Docker.

```
[ status: OK ]
manel@manel-virtual-machine:~$ curl -s http://127.0.0.1:5000/health
{"status":"ok"}
manel@manel-virtual-machine:~$ curl -s http://127.0.0.1:5000/
{"message":"Hello Kubernitest (Docker)"}
manel@manel-virtual-machine:~$
```

Capture d'écran de la requête HTTP vers l'endpoint / exécutée sur le conteneur Docker.
 Capture d'écran de la requête HTTP vers l'endpoint /health exécutée sur le conteneur Docker.

Ces tests confirment que l'application fonctionne de manière identique en local et dans un conteneur Docker.

6.5 Consultation des logs du conteneur

Les logs générés par l'application ont été consultés à l'aide de la commande `docker logs`. Ils permettent de vérifier le démarrage de l'application ainsi que les requêtes HTTP reçues.

```
(.venv) manel@manel-virtual-machine:~/kubernitest/app$ docker logs kubernitest-app
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [28/Jan/2026 15:22:26] "GET /health HTTP/1.1" 200 -
172.17.0.1 - - [28/Jan/2026 15:22:31] "GET / HTTP/1.1" 200 -
(.venv) manel@manel-virtual-machine:~/kubernitest/app$
```

```
172.17.0.1 - - [28/Jan/2026 15:22:26] "GET /health HTTP/1.1" 200 -
172.17.0.1 - - [28/Jan/2026 15:22:31] "GET / HTTP/1.1" 200 -
(.venv) manel@manel-virtual-machine:~/kubernitest/app$ docker stop kubernitest-app
kubernitest-app
(.venv) manel@manel-virtual-machine:~/kubernitest/app$
(.venv) manel@manel-virtual-machine:~/kubernitest/app$
```

Capture d'écran des logs du conteneur Docker.

L'accès aux logs constitue un élément essentiel pour le débogage et la supervision d'une application conteneurisée.

7.1 Présentation de Docker Compose

Docker Compose est un outil permettant de définir et gérer des applications multi-conteneurs à l'aide d'un fichier de configuration au format YAML.

Il permet de décrire les services, leurs ports, leurs variables d'environnement et leurs dépendances, puis de les démarrer avec une seule commande.

Dans ce projet, Docker Compose est utilisé afin d'orchestrer l'application localement, en préparation d'un déploiement futur sur Kubernetes.

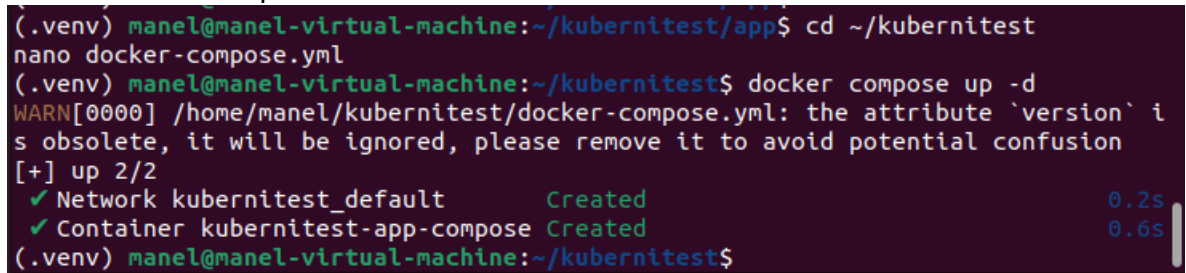
7.2 Définition du service applicatif

Un fichier docker-compose.yml a été créé à la racine du projet.

Il décrit un service unique correspondant à l'application conteneurisée précédemment avec Docker.

Le fichier permet notamment de :

- définir l'image Docker utilisée,
- exposer le port de l'application,
- définir les variables d'environnement nécessaires au fonctionnement de l'application,
- attribuer un nom explicite au conteneur.



```
(.venv) manel@manel-virtual-machine:~/kubernitest/app$ cd ~/kubernitest
nano docker-compose.yml
(.venv) manel@manel-virtual-machine:~/kubernitest$ docker compose up -d
WARN[0000] /home/manel/kubernitest/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 2/2
✓ Network kubernitest_default          Created          0.2s
✓ Container kubernitest-app-compose Created          0.6s
(.venv) manel@manel-virtual-machine:~/kubernitest$
```

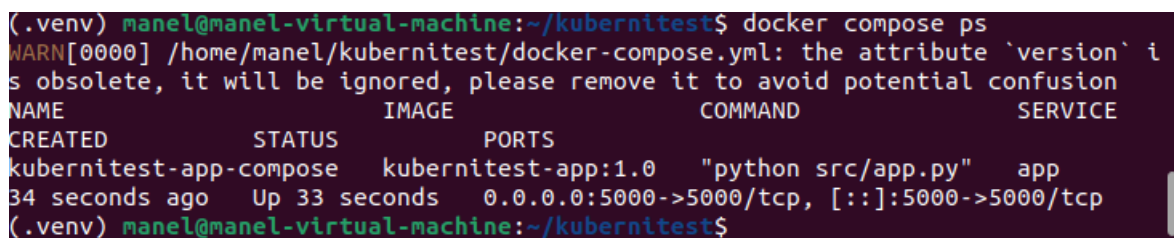
Capture d'écran montrant la présence du fichier docker-compose.yml.

7.3 Lancement de l'application avec Docker Compose

L'application a été lancée à l'aide de la commande `docker compose up -d`.

Cette commande permet de démarrer les services décrits dans le fichier de configuration en arrière-plan.

Une vérification a ensuite été effectuée afin de s'assurer que le service est bien en cours d'exécution.



```
(.venv) manel@manel-virtual-machine:~/kubernitest$ docker compose ps
WARN[0000] /home/manel/kubernitest/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE              COMMAND              SERVICE
CREATED            STATUS            PORTS
kubernitest-app-compose kubernitest-app:1.0 "python src/app.py" app
34 seconds ago     Up 33 seconds     0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
(.venv) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran de la commande `docker compose ps` confirmant que le service est actif.

7.4 Validation du fonctionnement de l'application orchestrée

Le bon fonctionnement de l'application a été validé une nouvelle fois via des requêtes HTTP effectuées depuis la machine hôte.

- L'endpoint / retourne le message attendu.
- L'endpoint /health confirme que l'application est opérationnelle.

```
manel@manel-virtual-machine:~$ curl -s http://127.0.0.1:5000/  
curl -s http://127.0.0.1:5000/health  
{"message":"Hello Kuberntest (Compose)"}  
{"status":"ok"}  
manel@manel-virtual-machine:~$
```

Capture d'écran de la requête HTTP vers l'endpoint / exécutée via Docker Compose.

Capture d'écran de la requête HTTP vers l'endpoint /health exécutée via Docker Compose.

Ces tests confirment que l'application fonctionne correctement lorsqu'elle est orchestrée via Docker Compose.

7.5 Consultation des logs avec Docker Compose

Les logs de l'application ont été consultés à l'aide de la commande `docker compose logs`. Ils permettent de vérifier le démarrage de l'application ainsi que les requêtes HTTP reçues par le conteneur.

```
(.env) manel@manel-virtual-machine:~/kubernitest$ docker compose logs  
WARN[0000] /home/manel/kubernitest/docker-compose.yml: the attribute `version` i  
s obsolete, it will be ignored, please remove it to avoid potential confusion  
kubernitest-app-compose | * Serving Flask app 'app'  
kubernitest-app-compose | * Debug mode: off  
kubernitest-app-compose | WARNING: This is a development server. Do not use it  
in a production deployment. Use a production WSGI server instead.  
kubernitest-app-compose | * Running on all addresses (0.0.0.0)  
kubernitest-app-compose | * Running on http://127.0.0.1:5000  
kubernitest-app-compose | * Running on http://172.18.0.2:5000  
kubernitest-app-compose | Press CTRL+C to quit  
kubernitest-app-compose | 172.18.0.1 - - [28/Jan/2026 15:45:22] "GET / HTTP/1.1  
" 200 -  
kubernitest-app-compose | 172.18.0.1 - - [28/Jan/2026 15:45:22] "GET /health HT  
TP/1.1" 200 -  
(.env) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran des logs de l'application affichés via Docker Compose.

L'accès centralisé aux logs constitue un avantage important de l'orchestration avec Docker Compose.

7.6 Arrêt de l'application

Enfin, l'ensemble des services a été arrêté proprement à l'aide de la commande `docker compose down`.

```
(.venv) manel@manel-virtual-machine:~/kubernitest$ docker compose down
WARN[0000] /home/manel/kubernitest/docker-compose.yml: the attribute `version` is
obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
✓ Container kubernitest-app-compose Removed 10.6s
✓ Network kubernitest_default Removed 0.2s
(.venv) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran montrant l'arrêt des services Docker Compose.

Cette commande permet de libérer les ressources et d'arrêter proprement l'application orchestrée.

8. Mise en place du cluster Kubernetes avec Kind

8.1 Vérification de l'outil Kind

Avant la création du cluster Kubernetes, une vérification de l'outil **Kind (Kubernetes IN Docker)** a été effectuée afin de s'assurer de sa disponibilité et de sa version.

```
(.venv) manel@manel-virtual-machine:~/kubernitest$ kind version
kind v0.23.0 go1.21.10 linux/amd64
```

Capture d'écran de la commande `kind version`.

Cette vérification confirme que Kind est correctement installé et prêt à être utilisé pour la création d'un cluster Kubernetes local.

8.2 Création du cluster Kubernetes local avec Kind

Un cluster Kubernetes local a été créé à l'aide de Kind.

Ce cluster repose sur des nœuds Kubernetes exécutés dans des conteneurs Docker.

```
(.venv) manel@manel-virtual-machine:~/kubernitest$ kind create cluster --name kubernitest
Creating cluster "kubernitest" ...
✓ Ensuring node image (kindest/node:v1.30.0)
✓ Preparing nodes
✓ Writing configuration
✓ Starting control-plane
✓ Installing CNI
✓ Installing StorageClass
Set kubectl context to "kind-kubernitest"
You can now use your cluster with:

kubectl cluster-info --context kind-kubernitest

Have a question, bug, or feature request? Let us know! https://kind.sigs.k8s.io/#community
(.venv) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran montrant la création du cluster Kubernetes avec la commande `kind create cluster`.

La création du cluster inclut :

- la préparation des nœuds,

- l'installation du plan de contrôle (control plane),
- la configuration automatique de kubectl pour utiliser le contexte du cluster créé.

8.3 Vérification du bon fonctionnement du cluster

Une fois le cluster créé, plusieurs vérifications ont été réalisées afin de s'assurer de son bon fonctionnement.

La commande kubectl cluster-info a permis de vérifier que le plan de contrôle Kubernetes est accessible et opérationnel.

```
(.env) manel@manel-virtual-machine:~/kubernitest$ kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:35469
CoreDNS is running at https://127.0.0.1:35469/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
(.env) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran de la commande kubectl cluster-info.

Ensuite, la commande kubectl get nodes a été utilisée pour vérifier l'état des nœuds du cluster.

```
(.env) manel@manel-virtual-machine:~/kubernitest$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
kubernitest-control-plane Ready    control-plane  2m52s  v1.30.0
(.env) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran montrant le nœud Kubernetes en état Ready.

Ces vérifications confirment que le cluster Kubernetes est fonctionnel et prêt à accueillir des déploiements applicatifs.

8.4 Création et vérification du namespace

Afin d'isoler les ressources de l'application au sein du cluster, un namespace dédié a été créé.

La création d'un namespace permet :

- de structurer les ressources Kubernetes,
- d'isoler l'application du reste du cluster,
- de faciliter l'administration et la lisibilité.


```
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
kubernitest-control-plane          Ready    control-plane  2m52s   v1.30.0
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl create namespace kubernitest
namespace/kubernitest created
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl get namespaces
NAME                STATUS    AGE
default             Active    3m22s
kube-node-lease     Active    3m22s
kube-public         Active    3m22s
kube-system         Active    3m22s
kubernitest         Active    6s
local-path-storage  Active    3m13s
(.venv) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran montrant la création du namespace kubernitest et la liste des namespaces disponibles.

8.5 Chargement de l'image Docker dans le cluster Kind

Par défaut, un cluster Kind ne dispose pas des images Docker présentes localement sur la machine hôte.

Il est donc nécessaire de charger manuellement l'image Docker de l'application dans le cluster.

Cette opération a été réalisée à l'aide de la commande `kind load docker-image`.

```
(.venv) manel@manel-virtual-machine:~/kubernitest$ kind load docker-image kubernitest-app:1.0 --name kubernitest
Image: "kubernitest-app:1.0" with ID "sha256:8232e7b05c1fa9f4e2d919d8a38249134c017b3788126ab5c1725b5a75c42790" not yet present on node "kubernitest-control-plane", loading...
(.venv) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran montrant le chargement de l'image Docker `kubernitest-app:1.0` dans le cluster Kubernetes.

Cette étape garantit que l'image de l'application est disponible pour les futurs déploiements Kubernetes.

Déploiement de l'application sur Kubernetes

9.1 Déploiement de l'application via un Deployment

L'application a été déployée sur le cluster Kubernetes à l'aide d'un **Deployment**.

Ce composant permet de gérer le cycle de vie des pods, d'assurer leur disponibilité et de faciliter les mises à jour applicatives.

Le fichier `deployment.yaml` a été appliqué au cluster à l'aide de la commande `kubectl apply`.

```
(.venv) manel@manel-virtual-machine:~/kubernitest$ nano k8s/deployment.yaml
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl apply -f k8s/deployment.yaml
deployment.apps/kubernitest-app created
(.venv) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran montrant l'application du **fichier deployment.yaml** et la création du **Deployment**.

Une vérification a ensuite été effectuée afin de s'assurer que le Deployment est correctement créé et que le pod associé est en cours d'exécution.

```
deployment.apps/kubernitest-app created
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl get deployments -n kubernitest
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
kubernitest-app     1/1     1             1           58s
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl get pods -n kubernitest
NAME                                     READY   STATUS    RESTARTS   AGE
kubernitest-app-5564d9bf58-965km       1/1     Running   0           63s
(.venv) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran des commandes **kubectl get deployments** et **kubectl get pods** montrant un pod en état Running et Ready.

Ces vérifications confirment que l'application est correctement déployée au sein du cluster Kubernetes.

9.2 Exposition de l'application via un Service Kubernetes

Afin de rendre l'application accessible, un **Service Kubernetes** de type **NodePort** a été créé.

Le Service permet d'exposer le port de l'application exécutée dans le pod vers l'extérieur du cluster.

Le fichier service.yaml a été appliqué avec succès.

```
(.venv) manel@manel-virtual-machine:~/kubernitest$ nano k8s/service.yaml
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl apply -f k8s/service.yaml
service/kubernitest-service created
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl get services -n kubernitest
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernitest-service NodePort    10.96.51.38   <none>        5000:30007/TCP   7s
```

Capture d'écran montrant la création **du Service Kubernetes** et la liste des services disponibles dans le namespace.

La vérification du Service confirme que :

- le type **NodePort** est bien utilisé,
- le port applicatif est correctement exposé.

9.3 Accès à l'application déployée sur Kubernetes

Dans le cadre d'un cluster Kubernetes local basé sur **Kind**, l'accès à l'application a été réalisé à l'aide de la commande `kubectl port-forward`. Cette méthode permet de rediriger un port local vers le port exposé par le Service Kubernetes.

```
<none> Debian GNU/Linux 12 (bookworm) 6.8.0-90-generic containerd:
//1.7.15
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl port-forward svc/kube
rnitest-service 30007:5000 -n kubernitest
Forwarding from 127.0.0.1:30007 -> 5000
Forwarding from [::1]:30007 -> 5000
```

Capture d'écran montrant la mise en place **du port-forward** entre la machine hôte et le Service **Kubernetes**.

Une fois le port-forward actif, des requêtes HTTP ont été effectuées afin de valider le bon fonctionnement de l'application.

```
manel@manel-virtual-machine:~$ curl http://127.0.0.1:30007/health
{"status":"ok"}
manel@manel-virtual-machine:~$ curl http://127.0.0.1:30007/
{"message":"Hello Kubernitest (Kubernetes)"}
manel@manel-virtual-machine:~$
```

Capture d'écran des requêtes HTTP vers **les endpoints / et /health**.

Les résultats montrent que :

- l'endpoint / retourne le message attendu,
- l'endpoint /health confirme que l'application est opérationnelle.

Ces tests valident l'accessibilité et le bon fonctionnement de l'application déployée sur Kubernetes.

9.4 Vérification de l'infrastructure Kubernetes

Des informations complémentaires ont été consultées afin de vérifier l'état de l'infrastructure Kubernetes, notamment les informations relatives au nœud du cluster.

```
kubernitest-service NodePort 10.96.51.38 <none> 5000:30007/TCP /s
(.venv) manel@manel-virtual-machine:~/kubernitest$ kubectl get nodes -o wide
NAME STATUS ROLES AGE VERSION INTERNAL-IP EXTERNAL-IP OS-IMAGE KERNEL-VERSION CONTAINER-R
NTIME
kubernitest-control-plane Ready control-plane 17m v1.30.0 172.18.0.2 <none> Debian GNU/Linux 12 (bookworm) 6.8.0-90-generic containerd:
/1.7.15
(.venv) manel@manel-virtual-machine:~/kubernitest$
```

Capture d'écran de la **commande kubectl get nodes -o wide** montrant un nœud en état Ready.

Cette vérification confirme que le cluster Kubernetes est sain et capable d'héberger l'application.

Conclusion générale

L'objectif de ce projet était de mettre en pratique les concepts de conteneurisation et d'orchestration à travers la réalisation d'une application simple, déployée progressivement depuis un environnement local jusqu'à un cluster Kubernetes.

Dans un premier temps, l'application a été développée et testée localement afin de valider son fonctionnement. Elle a ensuite été conteneurisée avec Docker, ce qui a permis de standardiser son exécution et de mieux comprendre la différence entre image et conteneur. L'utilisation de Docker Compose a facilité l'orchestration locale et la gestion de la configuration, tout en préparant la transition vers Kubernetes.

La mise en place d'un cluster Kubernetes local avec Kind a permis de découvrir le fonctionnement d'un environnement orchestré. Le déploiement de l'application via un Deployment, ainsi que son exposition à l'aide d'un Service, ont permis de comprendre la gestion du cycle de vie des pods, l'accès réseau et la supervision de l'application. Les probes de santé et la consultation des logs ont renforcé la fiabilité du déploiement et la capacité à diagnostiquer le fonctionnement de l'application.

Ce projet a permis de rencontrer et de résoudre plusieurs problématiques techniques, notamment liées à la configuration des outils et à l'accès à l'application dans un cluster local. Ces étapes ont contribué à une meilleure compréhension concrète de Docker et Kubernetes.

En conclusion, ce projet a permis d'atteindre les objectifs pédagogiques fixés et de consolider les bases nécessaires à la mise en œuvre d'applications conteneurisées et orchestrées. Il constitue une base solide pour aborder des architectures plus complexes intégrant plusieurs services ou des mécanismes avancés de déploiement.

